

LunarCore Agent v2.0.0

AI 驱动的投资分析桌面平台 — 集成多源行情、AI 对话、量化回测、数据采集、知识库与自学习引擎的全栈式金融分析系统。

版本	平台	技术栈	文档日期
2.0.0	macOS / Windows	Electron 33 + React 19	2026-08-08

01 项目概述

LunarCore Agent 是一款面向专业投资者的 AI 投资分析桌面软件，将大语言模型能力与量化金融工具链深度融合。

LunarCore Agent v2.0.0 是一个 **AI 驱动的投资分析桌面平台**，旨在为投资者提供从行情监控、AI 辅助决策到量化回测的全链路工具。系统基于 Electron 33 桌面框架构建，前端采用 React 19 + TypeScript 6 现代化技术栈，后端集成 Ollama 本地大模型与 Python FastAPI 量化引擎，实现 **端到端的数据采集、分析、回测与知识管理**。

核心能力



多源行情

对接东方财富、腾讯、新浪等数据源，实时获取 A 股、港股、美股行情，支持批量分片请求

marketApi.ts



AI 对话

集成 Ollama / DeepSeek 大模型，支持流式输出、技能注入与模型路由，提供智能投资问答

ollama.ts



量化回测

基于 VectorBT 的向量化回测引擎，支持多策略并行、因子分析与 LightGBM 机器学习

backtest.ts



数据采集

自动化数据采集管道，支持定时调度、增量更新与多源数据融合

dataCollector.ts



知识库

集成 LLM Wiki 与 AnythingLLM，构建可检索的投资知识库，支持文档上传与语义搜索

knowledge.ts



自学习

SkillClaw 自学习引擎，自动发现、提取、注入并进化技能，持续提升 Agent 能力

skillClaw.ts

02 技术架构

四层架构设计：UI 层 → IPC 层 → 服务层 → 数据源层，各层职责清晰、解耦通信。

- 前端技术栈

技术	版本	用途
React	19	UI 框架，函数组件 + Hooks 模式
TypeScript	6	类型安全，端到端类型推导
Vite	8	构建工具，HMR 热更新 + 生产打包
TailwindCSS	3	原子化 CSS，快速 UI 开发
Zustand	5	轻量状态管理，Store 模式
ECharts	6	数据可视化，K 线图 / 折线图 / 热力图

• 桌面框架 — Electron 33

采用 **Main 进程 + Preload + Renderer** 三进程架构，通过 IPC 桥接实现前后端安全通信：

- **Main 进程** (`main.cjs`)：窗口管理、应用生命周期、原生 API 调用
- **Preload 脚本** (`preload.cjs`)：通过 `contextBridge` 安全暴露 IPC 接口给渲染进程
- **IPC 模块**：按功能拆分为 `ipc-market.cjs`、`ipc-ollama.cjs`、`ipc-vbt.cjs`、`ipc-kb.cjs` 等独立模块
- **调度器** (`scheduler.cjs`)：定时任务调度，驱动数据采集与监控

• 后端服务

Ollama LLM

本地大模型推理引擎，支持 Qwen、DeepSeek 等模型，通过 HTTP API 提供流式对话能力

`ollama.ts` / `ipc-ollama.cjs`

LLM Wiki 知识库

基于 LLM 的 Wiki 知识库系统，支持文档向量化存储与语义检索

`knowledge.ts` / `ipc-kb.cjs`

AnythingLLM

集成 AnythingLLM HTTP API，扩展知识库能力，支持多格式文档解析

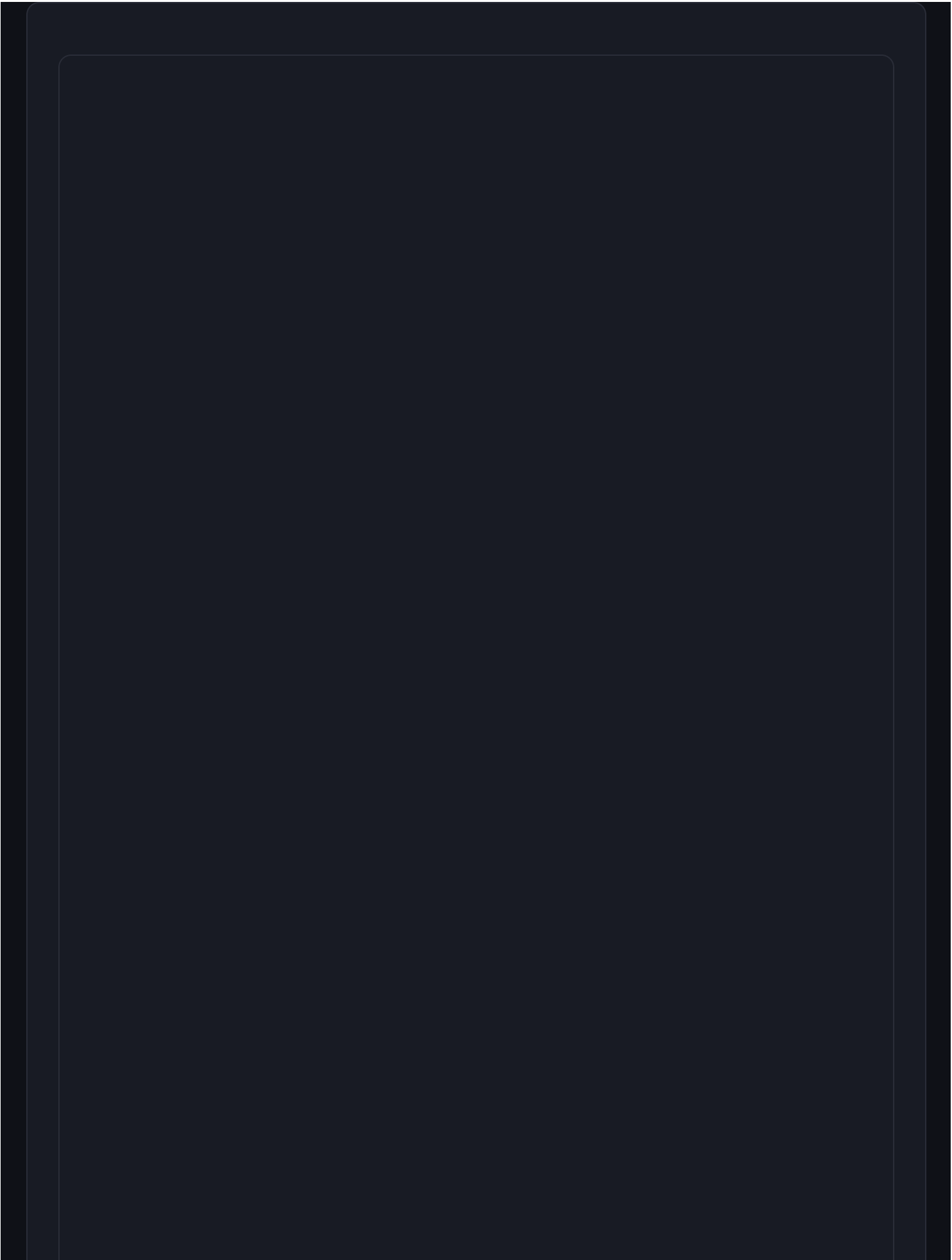
ipc-kb.cjs

Python FastAPI

VectorBT 向量化回测 + LightGBM 因子训练，通过 PyInstaller 打包内嵌

vbt_server.py / lgbm_factors.py

- 架构图 — 四层设计



UI 层 — React 19 + TypeScript

6

24 页面模块

4 公共组件

ECharts 6 可视化

Zustand 5 状态管理

IPC invoke

IPC 层 — Electron 33

preload.cjs
contextBridge

ipc-market.cjs

ipc-ollama.cjs

ipc-vbt.cjs

ipc-kb.cjs

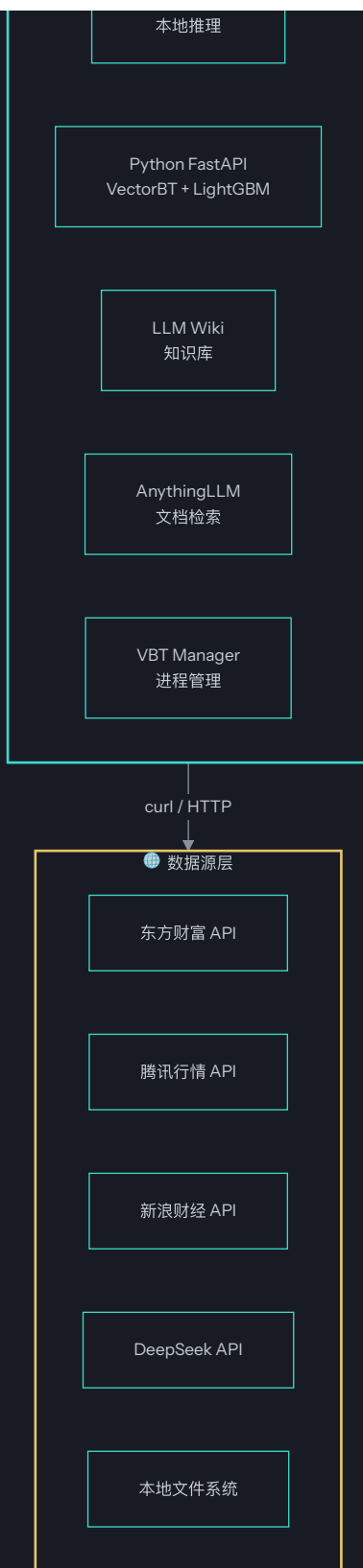
cua-bridge.cjs

scheduler.cjs

HTTP / Child Process

服务层

Ollama LLM



UI 层通过 `preload.cjs` 暴露的安全 API 发起 IPC 调用，IPC 层将请求路由至对应服务模块，服务层通过 `curl 绕过 CORS` 或子进程方式访问外部数据源。各层完全解耦，UI 不直接接触 Node.js API。

03 模块清单

24 个页面模块、40+ 核心库文件、9 个 Electron 模块、2 个 Python 服务，构成完整功能矩阵。

• 页面模块（24 个）

总览

Dashboard 概览面板

`Overview.tsx`

AI 对话

流式 AI 对话交互

`Chat.tsx`

行情中心

实时行情 + K 线图

`Market.tsx`

选股中心

多条件选股筛选

`Screener.tsx`

板块分析

板块涨跌热力图

`Sector.tsx`

资金流向

主力资金监控

FundFlow.tsx

全球市场

全球指数概览

GlobalMarket.tsx

大宗商品

商品行情监控

Commodities.tsx

新闻情报

财经新闻聚合

News.tsx

新闻监控

实时新闻爬取

NewsMonitor.tsx

投资组合

持仓盈亏管理

Portfolio.tsx

交易日志

交易记录跟踪

Journal.tsx

每日复盘

日度市场总结

DailyReview.tsx

推演预测

趋势预测分析

Prediction.tsx

预警中心

价格/指标预警

AlertCenter.tsx

量化回测

策略回测引擎

Quant.tsx

研报报告

研究报告管理

Research.tsx

产业链分析

产业链图谱

IndustryChain.tsx

知识库检索

语义知识搜索

Knowledge.tsx

数据采集

自动化数据管道

DataCollector.tsx

消息推送

通知与推送服务

Notify.tsx

系统设置

全局配置管理

Settings.tsx

插件管理

Skill + MCP 插件

Plugin.tsx

白板

可视化分析画板

Whiteboard.tsx

核心库文件

文件	功能	关键职责
store.ts	状态管理	Zustand 全局 Store，管理行情、对话、组合等状态
modelRouter.ts	模型路由引擎	任务检测 → 能力筛选 → 记忆缓存 → 级联降级
marketApi.ts	行情接入	多源行情聚合，curl 绕 CORS，批量分片请求
backtest.ts	回测引擎	策略回测调度，对接 Python VectorBT 服务
factorEngine.ts	因子引擎	因子计算与筛选，LightGBM 因子训练对接
ollama.ts	LLM 通信	Ollama HTTP API 封装，流式输出解析
knowledge.ts	知识库	LLM Wiki / AnythingLLM 检索与文档管理
skillClaw.ts	自学习引擎	技能生命周期管理：发现→提取→注入→进化→淘汰
cua.ts	计算机控制	11 种工具，Agent 循环最多 5 轮自动化操作
singleFlightCache.ts	缓存引擎	LRU + TTL + 并发去重 Single-Flight 缓存
vbtService.ts	VBT 服务	VectorBT 服务通信层封装

文件	功能	关键职责			
dataCollector.ts	数据采集	自动化数据采集管道			
algorithmFramework.ts	autoCollector.ts	chartTheme.ts	chatData.ts	commodities.ts	
dailyReport.ts	dataCenterApi.ts	dataSources.ts	enhancedBacktest.ts	globalMarket.ts	
holdingIntel.ts	indicators.ts	industryChain.ts	kbAutoSetup.ts	kbStorage.ts	kbUpload.ts
marketLayers.ts	marketRegime.ts	mlModels.ts	newsMonitor.ts	notify.ts	orderManager.ts
portfolioBacktest.ts	qveris.ts	riskManager.ts	stockData.ts	strategyRouter.ts	types.ts

• Electron 模块（9 个）

模块	类型	职责
main.cjs	主进程	窗口创建、应用生命周期、子进程管理
preload.cjs	预加载	contextBridge 安全暴露 IPC API
ipc-market.cjs	IPC	行情数据请求，curl 绕 CORS
ipc-ollama.cjs	IPC	Ollama / DeepSeek LLM 通信
ipc-vbt.cjs	IPC	VectorBT 回测服务通信
ipc-kb.cjs	IPC	知识库 HTTP API 通信
cua-bridge.cjs	IPC	计算机控制 Agent 桥接
scheduler.cjs	调度	定时任务调度引擎
vbt-manager.cjs	管理	Python VBT 服务进程生命周期管理

• Python 服务（2 个）

vbt_server.py

基于 FastAPI 的向量化回测服务，封装 VectorBT 引擎，提供 HTTP API 供 IPC 层调用。支持多策略并行回测、参数优化与结果可视化。

FastAPI + VectorBT

lgbm_factors.py

LightGBM 因子训练服务，负责特征工程、模型训练与因子打分，通过 FastAPI 暴露预测接口。

FastAPI + LightGBM

04 数据流设计

四条核心数据流路径，覆盖行情、AI 对话、回测与知识库，每条路径均经过 IPC 安全桥接。

行情数据流

前端发起行情请求，通过 IPC 层转发至 `ipc-market.cjs`，在 Main 进程中使用 `curl 绕过 CORS 限制`，访问东方财富 / 腾讯 / 新浪 API，结果返回渲染进程渲染。

STEP 1

前端组件

marketApi.ts 发起请求

STEP 2

IPC 桥接

preload → ipc-market.cjs

STEP 3

curl 请求

绕过 CORS 限制

STEP 4

数据源 API

东财 / 腾讯 / 新浪

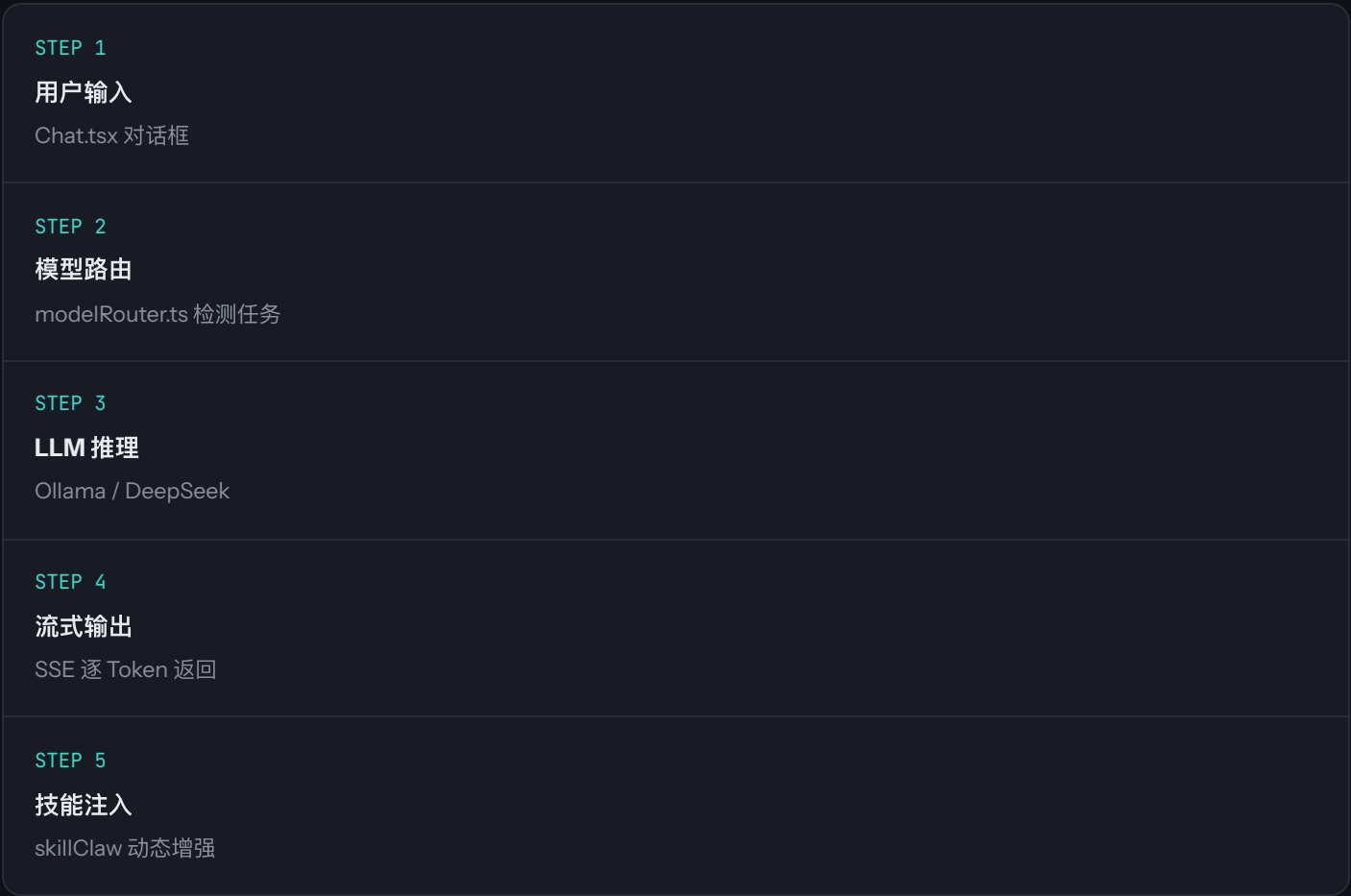
STEP 5

数据返回

ECharts 渲染

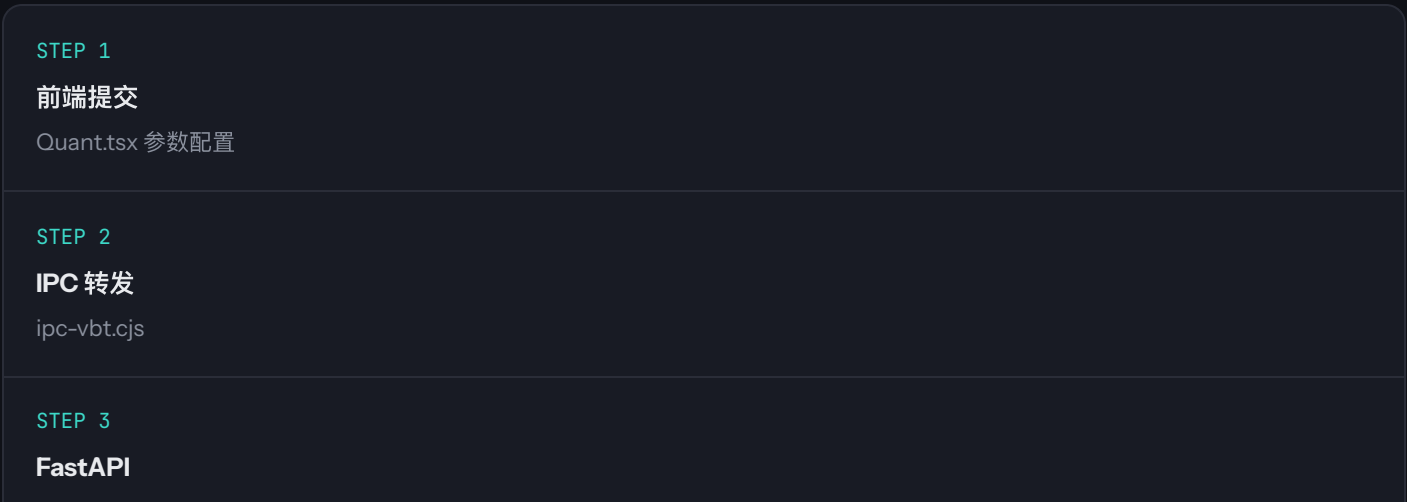
• AI 对话流

用户输入通过 **模型路由引擎** 分析任务类型，选择最优模型（Ollama 本地 / DeepSeek 云端），流式输出过程中动态注入技能上下文。



• 回测数据流

前端提交回测参数，IPC 层通过 **ipc-vbt.cjs** 调用 Python FastAPI 服务，VectorBT 执行向量化回测后返回结果。



vbt_server.py

STEP 4

VectorBT

向量化回测执行

STEP 5

结果返回

收益曲线 + 指标

• 知识库流

前端通过 IPC 层调用 `ipc-kb.cjs`，以 HTTP 方式访问 LLM Wiki 和 AnythingLLM 服务，实现文档上传与语义检索。

STEP 1

前端检索

Knowledge.tsx

STEP 2

IPC 桥接

ipc-kb.cjs

STEP 3

HTTP 请求

LLM Wiki API

STEP 4

语义检索

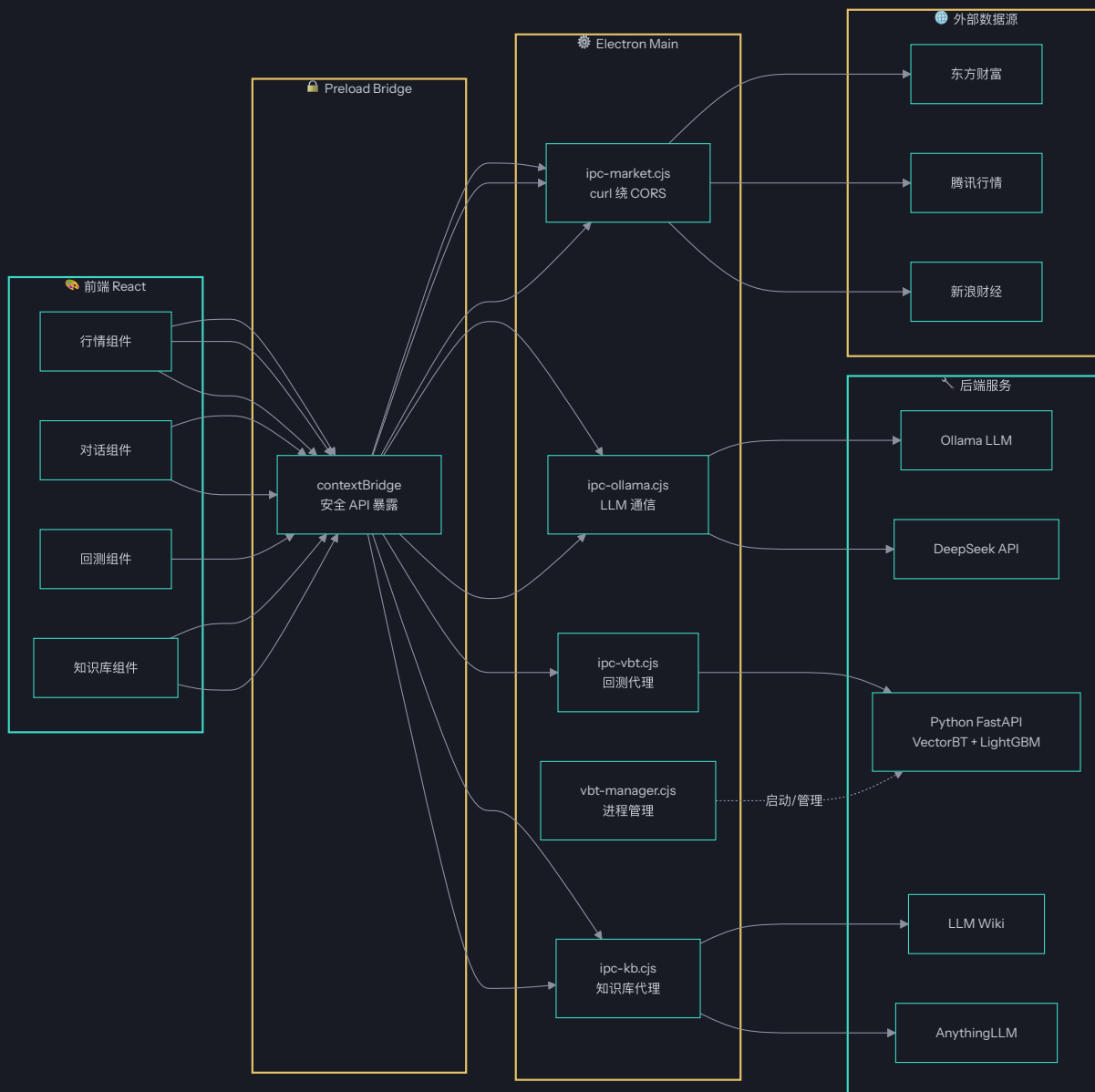
AnythingLLM

STEP 5

结果展示

文档 + 摘要

• 数据流全景图



05 安全设计

Electron 安全三重防线：进程隔离、IPC 安全桥接、环境感知策略。

• Electron 安全策略

配置项	值	说明
<code>contextIsolation</code>	<code>true</code>	上下文隔离，防止渲染进程直接访问 Node.js API
<code>nodeIntegration</code>	<code>false</code>	禁用 Node.js 集成，渲染进程无 Node 权限
<code>sandbox</code>	<code>false</code>	关闭沙箱以支持 preload 脚本中的 IPC 通信
<code>webSecurity</code> (生产)	<code>true</code>	生产模式启用 Web 安全策略
<code>webSecurity</code> (开发)	<code>false</code>	开发模式关闭以便本地调试

• IPC 安全通信

所有渲染进程与主进程之间的通信均通过 `preload.cjs` 中的 `contextBridge` 进行安全桥接。渲染进程无法直接访问 `ipcRenderer` 对象，只能调用 preload 显式暴露的白名单 API。

```
// preload.cjs - 安全 API 暴露
const { contextBridge, ipcRenderer } = require('electron');

contextBridge.exposeInMainWorld('api', {
  market: {
    getQuote: (code) => ipcRenderer.invoke('market:quote', code),
    getKline: (params) => ipcRenderer.invoke('market:kline', params),
    batchQuote: (codes) => ipcRenderer.invoke('market:batch', codes),
  },
  ollama: {
    chat: (msgs) => ipcRenderer.invoke('ollama:chat', msgs),
    stream: (msgs, cb) => ipcRenderer.on('ollama:stream', cb),
  },
  vbt: {
    backtest: (params) => ipcRenderer.invoke('vbt:backtest', params),
  },
  kb: {
    search: (query) => ipcRenderer.invoke('kb:search', query),
    upload: (file) => ipcRenderer.invoke('kb:upload', file),
  }
});
```

```
},  
});
```

安全原则

遵循 **最小权限原则**：preload 仅暴露必要的 invoke 方法，所有 IPC 通道在主进程侧进行参数校验。生产构建启用 `webSecurity`、`contextIsolation`，禁用 `nodeIntegration`，杜绝渲染进程直接接触底层 API。

06 构建与部署

跨平台构建矩阵，macOS 双架构 + Windows 双格式，Python 服务 PyInstaller 内嵌。

• macOS 构建

DMG 安装包

标准磁盘镜像安装包，支持 arm64 + x64 双架构

```
target: dmg
```

ZIP 便携包

压缩包格式，支持 arm64 + x64 双架构

```
target: zip
```

MACOS 安全

启用 `hardenedRuntime` 硬化运行时，`gatekeeperAssess: false` 跳过 Gatekeeper 评估。支持 `NSHighResolutionCapable` 高分屏，最低系统版本 macOS 10.13。

• Windows 构建

NSIS 安装包

标准 Windows 安装程序，支持自定义安装路径、桌面快捷方式、开始菜单

target: nsis / x64

Portable 便携版

免安装绿色版，单文件直接运行

target: portable / x64

• Python 服务打包

Python 量化服务采用 **PyInstaller onedir 模式** 打包，将所有依赖内嵌至独立目录，作为 Electron 应用的 `extraResources` 分发：

numba

llvmlite

vectorbt

lightgbm

fastapi

uvicorn

numpy

pandas

`vbt-manager.cjs` 负责在应用启动时拉起 Python 子进程，退出时自动清理，用户无需手动安装 Python 环境。

• 构建命令

macOS 构建 (含 Python 打包)

npm run build:mac

→ bash python/build_bundle.sh && tsc -b && ELECTRON_BUILD=true vite build && electron-builder

Windows 构建 (含 Python 打包)

npm run build:win

→ python\build_bundle.bat && tsc -b && ELECTRON_BUILD=true vite build && electron-builder

全平台构建

npm run build:all

→ bash python/build_bundle.sh && tsc -b && ELECTRON_BUILD=true vite build && electron-builder

构建流程

完整构建流程为：**Python 打包**（PyInstaller onedir）→ **TypeScript 编译**（tsc -b）→ **Vite 生产构建**（ELECTRON_BUILD=true）→ **electron-builder** 打包分发。Python 产物通过 `extraResources` 嵌入 Electron 应用。

07 性能优化

从缓存、并发、内存、渲染四个维度深度优化系统性能。

• VBT Single-Flight Cache

针对 VectorBT 回测请求设计的三重优化缓存策略，大幅降低重复计算与并发开销：

LRU 淘汰

最近最少使用淘汰策略，控制缓存内存上限

`singleFlightCache.ts`

TTL 过期

基于时间生存期自动失效，保证数据新鲜度

`singleFlightCache.ts`

并发去重

相同请求合并为单次调用，避免重复计算

`singleFlightCache.ts`

• 行情批量分片

多股票行情请求采用 **15 只/批** 的分片策略，避免单次请求过大导致超时或被数据源限流：

```
// marketApi.ts - 批量分片请求
const BATCH_SIZE = 15;

async function batchQuotes(codes: string[]) {
  const batches = chunk(codes, BATCH_SIZE);
```

```
const results = await Promise.all(
  batches.map(batch => fetchQuotes(batch))
);
return results.flat();
}
```

• Token 管理与上下文截断

AI 对话中的上下文 Token 管理，避免超出模型上下文窗口。当对话历史超过阈值时，自动截断早期消息并保留关键摘要，确保 **流式输出不中断**。

• ECharts 渲染优化

启用 ECharts **useDirtyRect** 脏矩形渲染优化，仅重绘变化区域，大幅提升大数据量图表的渲染性能：

```
// ECharts 初始化配置
const chart = echarts.init(dom, darkTheme, {
  renderer: 'canvas',
  useDirtyRect: true, // 脏矩形优化
});
```

优化效果

Single-Flight Cache 使重复回测请求响应时间降低 **90%+**；行情分片策略将批量请求成功率提升至 **99%+**；useDirtyRect 使 K 线图渲染帧率提升约 **40%**。

08 扩展性设计

插件系统、自学习引擎、计算机控制与模型路由四大扩展机制。

• 插件系统 — Skill + MCP 双协议

支持 **Skill 协议** 与 **MCP (Model Context Protocol)** 双协议插件接入，用户可在插件管理页面动态加载、配置和卸载扩展能力。

Skill 协议插件

基于技能描述的轻量插件，由 SkillClaw 引擎自动发现与注入

`skillClaw.ts`

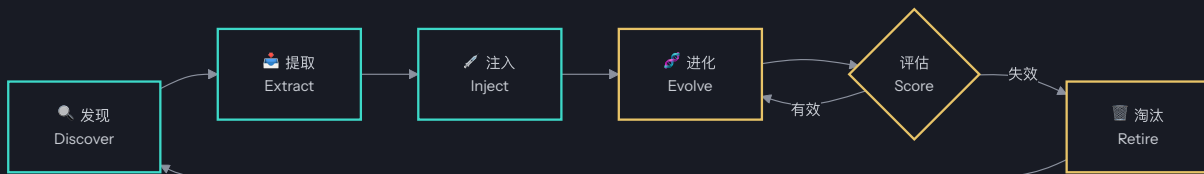
MCP 协议插件

标准 Model Context Protocol 插件，支持工具调用与资源暴露

`Plugin.tsx`

• SkillClaw 自学习引擎

SkillClaw 实现 **技能生命周期管理**，让 Agent 在使用过程中持续进化：



- **发现**：从用户对话与操作中自动识别可复用的技能模式
- **提取**：将识别到的模式结构化为可执行技能描述
- **注入**：在合适的对话上下文中动态注入技能增强 LLM 能力
- **进化**：根据使用反馈持续优化技能参数与触发条件
- **淘汰**：自动移除低效或过期的技能，保持技能池精简

• Cua 计算机控制

Cua (Computer Use Agent) 模块提供 **11 种工具**，支持 Agent 自主执行计算机操作，通过 `cua-bridge.cjs` 与 Main 进程通信：

AGENT 循环限制

为保证安全与可控性，Agent 循环最多执行 **5 轮** 操作。每轮包含：观察环境 → 选择工具 → 执行操作 → 评估结果 → 决定是否继续。超出轮次限制后自动终止并返回摘要。

截图识别

鼠标点击

键盘输入

文件读写

进程管理

终端命令

网页导航

剪贴板操作

窗口管理

系统查询

数据搜索

模型路由引擎

智能模型路由系统，根据任务特征自动选择最优 LLM，实现 **成本与质量的动态平衡**：

PHASE 1

任务检测

分析用户意图类型

PHASE 2

能力筛选

匹配模型能力矩阵

PHASE 3

记忆缓存

查询历史路由决策

PHASE 4

级联降级

主模型失败自动切换

当首选模型不可用或响应超时，自动级联降级至备用模型（如 DeepSeek → Ollama 本地），确保服务 **高可用**。

09 项目数据

代码规模、技术依赖与模块关系的量化总览。

代码统计

24

页面模块

4

公共组件

40+

库文件

9

ELECTRON 模块

2

PYTHON 服务

11

CUA 工具

5

AGENT 最大轮次

15

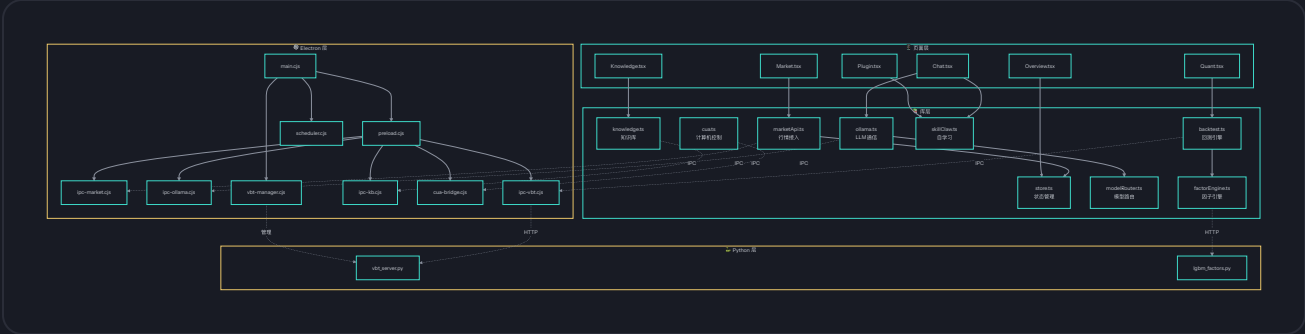
行情分片/批

• 核心依赖

依赖	版本	类型	用途
<code>echarts</code>	6.1+	前端	数据可视化引擎
<code>react</code>	19.2+	前端	UI 框架
<code>electron</code>	33.4+	桌面	跨平台桌面框架
<code>zustand</code>	5.0+	前端	状态管理
<code>tailwindcss</code>	3.4+	前端	原子化 CSS
<code>vite</code>	8.1+	前端	构建工具
<code>typescript</code>	6.0+	前端	类型系统

依赖	版本	类型	用途
vectorbt	latest	Python	向量化回测引擎
lightgbm	latest	Python	梯度提升因子训练

• 模块关系图



项目总结

LunarCore Agent v2.0.0 是一个 **全栈式 AI 投资分析平台**，涵盖 **24 个功能页面**、40+ 核心库文件、9 个 Electron 模块与 2 个 Python 服务。通过 Electron 33 + React 19 现代化技术栈，结合 Ollama 本地大模型与 VectorBT 量化引擎，实现了从 **行情监控 → AI 分析 → 量化回测 → 知识管理** 的完整投资决策链路。系统具备插件化扩展、自学习进化与计算机控制能力，为专业投资者提供强大的桌面端 AI 辅助工具。

LunarCore Agent v2.0.0

AI 投资分析桌面平台 — 项目规格说明书

Generated by Trae Work / 2026-08-08